МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту

Лабораторна робота №9

з дисципліни
«Операційні системи»

Виконав:
студент групи КН-114
Добрій Назарій

Викладач:
Кривенчук Ю.П.

Львів – 2020 р.

Тема. Виконання задачі в декількох потоках в ОС Linux

Мета. Навчитись реалізовувати розпаралелювання алгоритмів за допомогою багатопоточності в ОС Linux з використанням пакету функцій pthread.

Завдання.

1. Реалізувати заданий алгоритм в окремому потоці з використанням функцій з пакету pthread (завдання взяти з лабораторної роботи № 3).
2. Виконати розпаралелювання заданого алгоритму на 2, 4, 8 потоків.
3. Порівняти результати виконання програми під ОС Windows та Linux.
4. Результати виконання роботи відобразити у звіті.

Код реалізації програми:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <iostream>
#include <vector>
#include <ctime>

using namespace std;

pthread_t TIDs[8];

void* selectionSort(void* arg)
{
    vector<int>& temp = *(vector<int>* ) arg;

    int min = 0;
    for (int i = 0; i < temp.size() - 1; i++) {
        min = i;
        for (int j = (i + 1); j < temp.size(); j++)
            if (temp[j] < temp[min])
                min = j;
        swap( &: temp[i],  &: temp[min]);
    }

    pthread_exit( retval: 0);
}

vector<int> complete(vector<vector<int>> arr, int n, int k) {
    vector<int> completed(n);
    for (int i = 0; i < n; ++i) {
        int min = 1000000;
        int th = -1;
        for (int j = 0; j < k; j++) {
            if (!arr[j].empty() && arr[j][0] < min) {
                min = arr[j][0];
                th = j;
            }
        }
        completed[n - i - 1] = min;
        arr[th].erase(arr[th].begin());
    }
```

```
    return completed;
}

int main()
{
    srand(time(NULL));

    int amount_num, amount_thread;
    vector<vector<int>> arr;

    cout << "Enter the number of elements: ";
    cin >> amount_num;
    vector<int> input(amount_num);
    for (int i = 0; i < amount_num; i++) {
        input[i] = rand() % amount_num + 1;
        //cout << input[i] << ' ';
    }
    //cout << endl;

    cout << "Enter the number of threads: ";
    cin >> amount_thread;
    for (int i = 0; i < amount_thread; i++) {
        vector<int> tmp;
        for (int j = (i * amount_num / amount_thread); j < (i + 1) * amount_num / amount_thread; j++) {
            tmp.push_back(input[j]);
        }
        arr.push_back(tmp);
    }

    // Launch thread
    for (int i = 0; i < amount_thread; i++) {
        pthread_attr_t attr;
        pthread_attr_init(&attr);

        pthread_create(&TIDs[i], &attr, selectionSort, &arr[i]);

        cout << "TID = " << TIDs[i] << endl;
    }

```

```
    // Wait until thread is done its work
    for (int i = 0; i < amount_thread; i++) {
        pthread_join(TIDs[i], NULL);
    }

    vector<int> completed = complete(arr, amount_num, amount_thread);
    cout << "Sorted array: " << endl;
    for (int i = 0; i < amount_num; ++i) {
        cout << completed[i] << ' ';
    }
    cout << endl;

    return 0;
}
```

Реалізація програми у console:

```
Run: lab9
/home/lime/CLionProjects/lab9/cmake-build-debug/lab9
Enter the number of elements: 40
Enter the number of threads: 3
TID = 139701724829440
TID = 139701716436736
TID = 139701708044032
Sorted array:
39 38 35 35 35 35 34 34 33 33 32 32 31 30 28 28 26 24 24 23 20 20 17 16 16 15 13 13 12 10 10 9 8 6 5 4 3 3 2 2

Process finished with exit code 0
```

Запущені потоки:

lime@lime

Command Field Option Help

Linear Tree Thread Filter All Processes

PID	TTY	USER	STAT	MEM	%CPU	START	TIME	COMMAND_LINE
5161	-	lime	S	1.5M	0.00	22:07	0.00s	pool-gsd-media- (thread)
5159	-	lime	R	92.4M	24.95	22:06	29s	lab9 (thread)
5158	-	lime	R	92.4M	24.95	22:06	29s	lab9 (thread)
5157	-	lime	R	92.4M	24.95	22:06	29s	lab9 (thread)
5032	-	lime	S	92.4M	0.00	22:06	4.07s	lab9
5031	-	lime	S	371.8M	0.00	22:06	0.01s	PtyProcess Reap (thread)
5015	-	lime	S	374.3M	0.00	22:06	0.03s	ApplicationImpl (thread)
4983	-	root	IW	0	0.00	22:05	0.02s	(kworker/u256:1-events_unbound)
4971	-	lime	S	371.8M	0.00	22:05	0.01s	BaseDataReader: (thread)
4969	-	lime	S	371.8M	0.00	22:05	0.05s	BaseDataReader: (thread)
4966	-	lime	S	371.8M	0.00	22:05	0.02s	lab9 (thread)
4963	-	lime	S	371.8M	0.00	22:05	0.01s	ApplicationImpl (thread)
4962	-	lime	S	371.8M	0.00	22:05	0.08s	ApplicationImpl (thread)
4961	-	lime	S	371.8M	0.00	22:05	0.03s	ApplicationImpl (thread)
4959	pts/0	lime	S	5.8M	0.00	22:05	0.00s	gdbus (thread)
4958	pts/0	lime	S	5.8M	0.00	22:05	0.00s	gmain (thread)
4957	pts/0	lime	S	6.3M	0.00	22:05	0.08s	QXcbEventQueue (thread)
4956	pts/0	lime	S	6.3M	0.00	22:05	0.15s	QDBusConnection (thread)
4955	pts/0	lime	R	6.3M	0.96	22:05	5.97s	qps
4936	pts/0	lime	S	1.4M	0.00	22:05	0.02s	bash
4935	-	lime	S	4.4M	0.00	22:05	0.00s	threaded-ml (thread)
4931	-	lime	S	4.4M	0.00	22:05	0.00s	dconf worker (thread)
4930	-	lime	S	4.4M	0.00	22:05	0.00s	gdbus (thread)
4929	-	lime	S	4.4M	0.00	22:05	0.00s	gmain (thread)
4928	-	lime	S	4.4M	0.00	22:05	0.70s	gnome-terminal-server
4855	-	root	IW	0	0.00	22:03	0.00s	(kworker/3:2-events)
4803	-	root	IW	0	0.00	22:03	0.00s	(kworker/2:2)
4742	-	root	IW	0	0.00	22:02	0.00s	(kworker/0:2)
4729	-	root	IW	0	0.00	22:02	0.20s	(kworker/0:1-events)
4726	-	root	IW	0	0.00	22:02	0.01s	(kworker/3:1-events)
4724	-	lime	S	371.8M	0.00	22:02	0.07s	JobScheduler FJ (thread)
4723	-	lime	S	371.8M	0.00	22:02	0.05s	JobScheduler FJ (thread)
4721	-	lime	S	371.8M	0.00	22:02	0.48s	JobScheduler FJ (thread)

Process count: 640

Висновок: я навчився реалізовувати розпаралелювання алгоритмів за допомогою багатопоточності в ОС Linux з використанням пакету функцій pthread.